Assignment 2 (Weeks 3 and 4)

Solutions

Part I: Short Answer / Reasoning Questions

Q1. NULL comparison and aggregate exclusion

The query fails because phone_number = NULL is never TRUE. In SQL's three-valued logic (TRUE / FALSE / UNKNOWN), NULL represents an unknown or missing value, not a specific value that can be compared with "=". Comparing anything to NULL ,even NULL to NULL, evaluates to UNKNOWN, not TRUE. A WHERE clause only keeps rows where the condition evaluates to TRUE, so rows with a NULL phone_number are silently excluded, and the count is computed only over the (empty) set of rows where the comparison happened to be TRUE.

Corrected query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

In general, NULL is excluded from most aggregate computations and comparisons because it does not represent a value at all — it represents the absence of one. Allowing arithmetic or equality comparisons to treat NULL as a normal value would produce misleading results (e.g., averaging in an "unknown" as if it were zero or some real number). So SQL defines comparison operators on NULL to return UNKNOWN, and defines aggregate functions such as SUM, AVG, MIN, MAX, and COUNT(column) to simply skip NULL values, since there is nothing meaningful to add into the computation. COUNT(*) is the one exception, since it counts rows rather than evaluating a column's value.

Q2. Is DISTINCT redundant with GROUP BY?

Yes, the student is correct ; DISTINCT is redundant in this specific query. GROUP BY department already collapses the Employee rows into one group per distinct department value, and the SELECT list returns exactly one row per group (department, COUNT(*)). Because department is already unique across the rows produced by the GROUP BY, no two rows in the result set can be identical, so DISTINCT has nothing left to remove. Conceptually, DISTINCT here would scan an already-unique result set looking for duplicate (department, count) pairs and find none ,it changes nothing about the output, it only potentially adds a small, unnecessary processing step for the engine.

Q3. INNER JOIN vs LEFT JOIN row-count difference

If the LEFT JOIN (Customer as the left table) returns more rows than the INNER JOIN, that tells you there are customers in the Customer table who have no matching row in Order ,that is customers who have never placed an order, or whose foreign key linkage to Order simply doesn't exist for any row. INNER JOIN only keeps rows where a match exists on both sides, so it silently drops these customers; LEFT JOIN preserves every Customer row regardless of whether a match exists, filling the Order-side columns with NULL when there's no matching order.

Concretely, the scenario that produces extra LEFT JOIN rows is: one or more customers exist in Customer who do not appear as customer_id in any row of Order (e.g., a newly registered customer who browsed but never checked out). Each such customer contributes exactly one extra row to the LEFT JOIN result (with NULL order columns) that simply does not appear in the INNER JOIN result.

Q4. Invalid GROUP BY with non-aggregated, non-grouped column

The query is invalid because employee_name is neither part of the GROUP BY list nor wrapped in an aggregate function. Standard SQL requires that every column in the SELECT list be either (a) a grouping column, or (b) the input to an aggregate function, because the query groups rows at the department level, collapsing potentially many employee rows into a single output row per department.

Conceptually, if SQL allowed this to run as written, then for any department with more than one employee, the engine would have to pick one specific employee_name to display next to that department's average salary — but there is no principled basis for choosing which employee's name "represents" the whole group. The result would be arbitrary and non-deterministic (different runs, or different execution plans, could legitimately return different names), and it would falsely imply a one-to-one relationship between a department and a single employee that does not actually exist in the data.

Correction — if only the per-department average is wanted:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

If the intent was instead to show every employee alongside their department's average (without collapsing rows), a window function should be used so each employee keeps their own row:

```
SELECT department, employee_name,
AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
FROM Employee;
```

Q5. WHERE cannot filter on an aggregate

The error is using WHERE to filter on AVG(salary), an aggregate value that does not exist yet at the point WHERE is evaluated. SQL's logical execution order is roughly: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. WHERE filters individual rows before any grouping or aggregation happens, so at that stage there is no per-department average yet to compare against — only raw, ungrouped employee rows. Aggregates like AVG() are only computed once GROUP BY has formed the groups, and the appropriate place to filter on the result of an aggregate is HAVING, which runs after grouping.

Corrected query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Correlated vs non-correlated subqueries

(a) Products priced above the overall average product price — a *non-correlated* subquery is appropriate, because the inner query (the overall average price across all products) does not depend on any value from the outer row. It is a single, fixed scalar that can be computed once and reused for every comparison.

```
SELECT *
FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Employees who earn more than the average salary of their own department — a *correlated* subquery is appropriate, because the inner average must be recomputed per department, and the department to average over depends on the outer row currently being evaluated.

```
SELECT *
FROM Employee e
WHERE salary > (
    SELECT AVG(salary) FROM Employee e2 WHERE e2.department =
e.department
);
```

Key performance implication: a non-correlated subquery is evaluated once, and its single result is reused for every row of the outer query. A correlated subquery, by contrast, is logically re-evaluated once per outer row (conceptually a nested-loop pattern), since its result depends on values from that row. When the outer table is large, this can mean the inner aggregation runs a very large number of times, which is far more expensive than a query that can compute the needed aggregates once (e.g., via a join to a pre-aggregated derived table, or a window function) and reuse them — most query optimizers try to rewrite correlated subqueries this way internally, but it is not always possible, so correlated subqueries remain a common performance bottleneck on large tables.

Part II: Long Answer Question

Schema reused from Assignment 1:

```
Customer(customer_id, name, email, phone)
Restaurant(restaurant_id, name, cuisine_type, rating)
MenuItem(item_id, restaurant_id, item_name, price)
Order(order_id, customer_id, restaurant_id, order_date, status)
OrderItem(order_id, item_id, quantity)
```

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Equivalent NOT EXISTS form:

```
SELECT c.customer_id, c.name
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1 FROM "Order" o WHERE o.customer_id = c.customer_id
);
```

Why: An INNER JOIN would only keep customers who do have at least one matching order, which is the exact opposite of what we want — it cannot express "never". A LEFT JOIN keeps every Customer row regardless of a match, filling order columns with NULL when none exists, so filtering on order_id IS NULL isolates exactly the unmatched customers. NOT EXISTS achieves the same result and is generally preferred over NOT IN here, since NOT IN behaves incorrectly (returns no rows at all) if the subquery's column can contain NULL, whereas NOT EXISTS has no such pitfall.

(b) Average order value per restaurant, restaurants with more than 5 orders

```
SELECT r.restaurant_id, r.name, AVG(ov.order_value) AS avg_order_value
FROM Restaurant r
JOIN (
    SELECT o.order_id, o.restaurant_id,
           SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
) ov ON r.restaurant_id = ov.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(ov.order_id) > 5;
```

Why: An order's value itself requires a first-level GROUP BY (summing price × quantity per order_id), and the restaurant-level average requires a second-level GROUP BY over those order values — this two-stage aggregation is naturally expressed with a subquery (derived table) nested inside the outer query, since a single GROUP BY cannot aggregate at two different granularities at once. Inner JOINs are used because we only want orders that actually contain items and restaurants that actually have qualifying orders. HAVING (not WHERE) is used for the "more than 5 orders" condition because that condition is on COUNT(ov.order_id), an aggregate computed only after grouping; WHERE cannot reference it.

(c) Restaurants whose average order value exceeds the platform-wide average

```
WITH OrderValues AS (
    SELECT o.order_id, o.restaurant_id,
           SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
),
RestaurantAvg AS (
    SELECT restaurant_id, AVG(order_value) AS avg_order_value, COUNT(*)
AS order_count
    FROM OrderValues
```

```
    GROUP BY restaurant_id
    HAVING COUNT(*) > 5
)
SELECT ra.restaurant_id, ra.avg_order_value
FROM RestaurantAvg ra
WHERE ra.avg_order_value > (
    SELECT AVG(avg_order_value) FROM RestaurantAvg
);
```

Why a non-correlated subquery is enough: the platform-wide average is a single scalar value that does not depend on which restaurant row is currently being evaluated — it is the same fixed number for every comparison, so it can be computed once (here, as a subquery over the RestaurantAvg CTE, effectively nesting part (b)'s logic) and reused. A correlated subquery is unnecessary because there is no per-row dependency to express.

Note: This computes the platform-wide average as the average of each qualifying restaurant's average order value, weighting every restaurant equally regardless of order volume. An alternative definition — the average order value across all individual qualifying orders, which implicitly weights busier restaurants more heavily — would instead be computed as SELECT AVG(order_value) FROM OrderValues. Either definition is defensible; the query above should be read together with this assumption stated explicitly.

(d) Most ordered menu item (by total quantity) per restaurant

```
WITH ItemTotals AS (
    SELECT mi.restaurant_id, mi.item_id, mi.item_name,
           SUM(oi.quantity) AS total_quantity
    FROM MenuItem mi
    JOIN OrderItem oi ON mi.item_id = oi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
),
MaxPerRestaurant AS (
    SELECT restaurant_id, MAX(total_quantity) AS max_quantity
    FROM ItemTotals
    GROUP BY restaurant_id
)
SELECT it.restaurant_id, it.item_name, it.total_quantity
FROM ItemTotals it
JOIN MaxPerRestaurant m
  ON it.restaurant_id = m.restaurant_id
 AND it.total_quantity = m.max_quantity;
```

Why: total quantity per item is first computed per restaurant (ItemTotals), then the maximum of that total is found per restaurant (MaxPerRestaurant), and the two are joined back together to recover which item achieved that maximum. This avoids a correlated subquery by separating "compute the max" from "find the row(s) matching the max" into two simple grouped aggregations joined together.

Limitation: this GROUP BY + MAX + self-join pattern returns every item that ties for the maximum quantity at a restaurant, so a restaurant with a tie produces more than one row instead of a single "winner" — basic GROUP BY + MAX has no built-in way to

break ties or limit to exactly one row. It also does not generalize cleanly to a true "top-N per group" problem (e.g., top 3 items per restaurant), since that would require repeating the max-and-rejoin logic N times. A RANK() or ROW_NUMBER() window function with PARTITION BY restaurant_id ORDER BY total_quantity DESC handles both ties and arbitrary N far more directly, and is generally the preferred approach for top-N-per-group problems.

Part III: Normalization

Enrollment(student_id, student_name, course_id, course_name, instructor_id, instructor_name, instructor_office, grade, enrollment_date)

Q1. Functional dependencies implied by the scenario

- student_id → student_name
- course_id → course_name, instructor_id
- instructor_id → instructor_name, instructor_office
- course_id → instructor_name, instructor_office (transitively, via instructor_id)
- (student_id, course_id) → grade, enrollment_date

Q2. Is this table in 1NF?

Yes. 1NF requires that every column hold a single, atomic value (no repeating groups, no multivalued or composite attributes stuffed into one column) and that rows be uniquely identifiable. Each attribute here (student_name, course_name, grade, instructor_office, etc.) is a single atomic value per row, and (student_id, course_id) uniquely identifies each row, so the table satisfies 1NF. Its problems are redundancy and the anomalies that come with it — issues that 1NF does not address and that 2NF/3NF are specifically designed to fix.

Q3. Partial dependency violating 2NF

The candidate key is the composite (student_id, course_id), but student_name depends only on student_id, and course_name and instructor_id depend only on course_id — in each case on only part of the composite key, not the whole of it. These are partial dependencies, which violate 2NF (every non-key attribute must depend on the entire key, not a subset of it).

This causes real anomalies. Update anomaly: if a student's name changes, it must be updated on every enrollment row for every course that student is in; missing even one row leaves inconsistent names for the same student_id. Insertion anomaly: a new course and its name/instructor cannot be recorded at all until at least one student enrolls in it, since course_name only exists attached to an enrollment row. Deletion anomaly: if the last (and only) student enrolled in a course has their enrollment row deleted, the course_name and instructor information for that course is lost entirely along with it.

Q4. Transitive dependency violating 3NF

course_id → instructor_id → instructor_name, instructor_office. Here, instructor_name and instructor_office depend on course_id only transitively, through the non-key attribute instructor_id, rather than depending directly on the key. This is a transitive dependency, which violates 3NF (no non-key attribute should depend on another non-key attribute).

Anomaly: if an instructor moves to a new office, every enrollment row for every course they teach must be updated; if even one row is missed, the same instructor_id ends up associated with two different offices in the data, an inconsistency with no way to tell which is correct. There is also an insertion anomaly (an instructor's office cannot be recorded until they are assigned to a course that has at least one enrollment) and a deletion anomaly (deleting the last enrollment row for an instructor's only course erases their office information).

Q5. Decomposition into 3NF

```
Student(student_id, student_name)
```

Justified by: student_id → student_name. Separated out because student_name depended on only part of the original composite key.

```
Instructor(instructor_id, instructor_name, instructor_office)
```

Justified by: instructor_id → instructor_name, instructor_office. Separated out to remove the transitive dependency of these attributes on course_id.

```
Course(course_id, course_name, instructor_id (FK → Instructor))
```

Justified by: course_id → course_name, instructor_id. Separated out because course_name and instructor_id depended on only part of the original composite key.

```
Enrollment(student_id, course_id, grade, enrollment_date, (FKs →
Student, Course))
```

Justified by: (student_id, course_id) → grade, enrollment_date — the one dependency that genuinely needs the full composite key, so it remains as the associative table linking students and courses.

Part IV: Design Justification

In Part II(a), the "customers who never ordered" query could equally be written as a LEFT JOIN with an IS NULL filter or as a NOT EXISTS correlated subquery; I chose the LEFT JOIN form because it most directly mirrors the relational intent — "keep every customer, then keep only the ones with no match" — and reads naturally when the result needs additional Customer columns. Both forms are correctness-safe with respect to NULLs (unlike NOT IN, which silently breaks if the subquery's column contains NULL), and most optimizers rewrite both into the same anti-join plan, so performance is comparable; NOT EXISTS becomes preferable mainly when the unmatched condition itself is more elaborate than a simple key comparison.

In Part III, decomposing Enrollment into Student, Instructor, Course, and Enrollment removes the redundancy that caused update, insertion, and deletion anomalies, but it makes a common query — "show a student's name, course, instructor, and office together" — require three or four joins instead of a single table scan, and that join cost

grows with table size and query frequency. Real systems sometimes intentionally keep denormalized tables (or build them as reporting tables or materialized views) on read-heavy, join-heavy paths, deliberately trading some controlled redundancy and extra update work for lower query latency, while still keeping a normalized form as the system of record.